

Propiedades vs. Restricciones (Constraints)

A menudo, los términos "**propiedades**" y "**restricciones**" se usan indistintamente, pero en SQL tienen roles distintos:

- **Restricciones (Constraints):** Son reglas que **limitan** los datos que puedes insertar en una columna para asegurar su validez e integridad. Son como los candados de seguridad de tu base de datos.
- **Propiedades (o Atributos):** Son características que **definen el comportamiento** de una columna, como la forma en que se generan o almacenan los datos.

Principales Restricciones (Constraints)

Las restricciones son la base para el diseño de bases de datos relacionales robustas.

1. PRIMARY KEY (Llave Primaria)

Es la restricción más importante. Combina dos reglas:

- **UNIQUE:** Cada valor en la columna debe ser único.
- **NOT NULL:** La columna no puede tener valores nulos.

Uso: Identifica de forma única cada registro en una tabla. Solo puede haber una PRIMARY KEY por tabla.

Ejemplo:

```
CREATE TABLE estudiantes (  
    id_estudiante INT PRIMARY KEY,  
    nombre VARCHAR(50)  
);
```

En este caso, id_estudiante no se puede duplicar ni dejar vacío.

2. FOREIGN KEY (Llave Foránea)

Es la clave para crear relaciones entre tablas. Vincula una columna de una tabla (la secundaria) a la PRIMARY KEY de otra tabla (la principal). Esto asegura la integridad referencial, evitando que crees registros "huérfanos".

Uso: Modelar relaciones de la vida real, como "un cliente hace muchos pedidos".

Ejemplo:

```
CREATE TABLE pedidos (  
    id_pedido INT PRIMARY KEY,  
    id_cliente INT,  
    fecha DATE,  
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

Aquí, un id_cliente en la tabla pedidos solo puede existir si ya está presente en la tabla clientes.

3. NOT NULL

Asegura que una columna siempre debe contener un valor, no se puede dejar en blanco.

Uso: Para campos esenciales como nombres, correos electrónicos o contraseñas.

Ejemplo:

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL  
);
```

4. UNIQUE

Garantiza que todos los valores en la columna son únicos. A diferencia de la PRIMARY KEY, una tabla puede tener varias restricciones UNIQUE.

Uso: Para campos que deben ser únicos pero no son la clave principal, como un correo electrónico o un número de seguro social.

Ejemplo:

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE  
);
```

5. CHECK

Permite definir una condición personalizada que cada valor en la columna debe cumplir.

Uso: Para validar que los datos cumplan con una regla de negocio.

Ejemplo:

```
CREATE TABLE productos (  
    id INT PRIMARY KEY,  
    precio DECIMAL(10, 2) CHECK (precio > 0)  
);
```

Esta restricción asegura que el precio nunca sea cero o negativo.

Principales Propiedades (o Atributos)

Estas propiedades son la diferencia entre un diseño de base de datos eficiente y uno que requiere mucho trabajo manual.

1. AUTO_INCREMENT

Es la propiedad más utilizada. Genera automáticamente un número secuencial único para cada nuevo registro, lo que es perfecto para las llaves primarias. Elimina el riesgo de duplicados y el trabajo de asignar un ID manualmente.

Uso: Con una PRIMARY KEY numérica.

Ejemplo:

```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50)  
);
```

Cuando insertes un nuevo usuario, el id se asignará solo.

2. DEFAULT

Permite especificar un valor por defecto que se usará en la columna si no se proporciona uno explícitamente en la consulta INSERT.

Uso: Para campos opcionales que necesitan un valor inicial, como la fecha de registro o un estado predeterminado.

Ejemplo:

```
CREATE TABLE eventos (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    fecha DATE DEFAULT CURRENT_DATE()  
);
```

Si no proporcionas una fecha, se usará la fecha actual.

Resumen Rápido

Característica	Propósito	Ejemplo
PRIMARY KEY	Identificador único	id INT PRIMARY KEY
FOREIGN KEY	Relaciona tablas	FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
NOT NULL	Fuerza a tener un valor	nombre VARCHAR(50) NOT NULL
UNIQUE	Asegura valores únicos	email VARCHAR(100) UNIQUE
CHECK	Valida con una condición	CHECK (precio > 0)
AUTO_INCREMENT	Genera IDs automáticos	id INT AUTO_INCREMENT
DEFAULT	Asigna un valor por defecto	estado VARCHAR(20) DEFAULT 'activo'